

Biot Android App – Software Component Detailed Design (SWCDD)

Table of Contents

1. Scope	1
2. Context	2
3. Architecture Overview	2
3.1. Main Components and Responsibilities	2
3.2. Key Design Decisions	3
3.3. Interfaces / APIs	3
3.4. MQTT Broker Setup (Required Before Running App)	3
3.5. MQTT Connection Workflow	5
3.6. Processing Incoming Messages	5
3.7. Database Integration	5
3.8. Events	8
3.9. DBAccess	8
3.10. Class Diagram	9
4. Component and Class Overview	9
4.1. Main Classes and Responsibilities	10
4.2. Database Structure	10
4.3. Simulation and Testing	10
5. Runtime & Deployment	10
5.1. Running the Android App	11
6. Quality & Constraints	12
7. How-To (for developers)	13
7.1. Build & Run Locally	13
7.2. Configure (env vars, settings)	13
7.3. Testing basics	13
7.4. Testing	14
8. Open Questions / Risks	16
9. Appendix	16

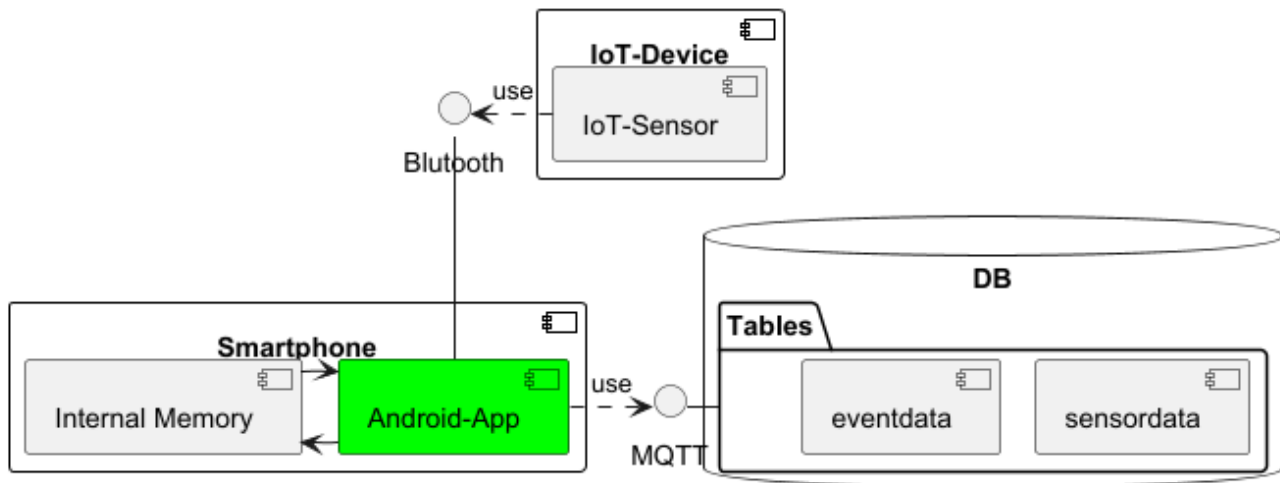
1. Scope

- Purpose: This project is an Android IoT application that receives MQTT sensor messages and displays them in real time.

- Goals: Real-time sensor data visualization, modular and maintainable code.
- Non-goals: No cloud backend
- How to run: See setup and deployment instructions below.

2. Context

Component Diagram:



This IoT application listens for incoming MQTT sensor messages and displays them in real time inside the Android UI.

It uses:

- MQTT (Eclipse Paho) for sensor communication
- Room Database for storing incoming sensor values
- Android TextViews for live output of acceleration, gyro, magnet, and time
- Douglas-Peuker algorithm for data reduction

3. Architecture Overview

3.1. Main Components and Responsibilities

- **MainActivity**: Central controller for UI, MQTT, and database
- **MqttHandler**: Handles MQTT connection, subscription, publishing, and message forwarding
- **BaseChartActivity**: Base class for all chart activities, centralizes chart logic
- **AccelActivity**, **GyroActivity**, **MagnetActivity**: Display and filter respective sensor data
- **MainGraphActivity**: Overview chart, filtering, LiveData management
- **EreignisActivity**, **NewEreignisActivity**: Event management, RecyclerView adapters
- **SensorEreignis**: Threshold monitoring, notifications

- DatePickerHandler: Date selection for filtering
- DouglasPeukerAlg: Data reduction for charts

3.2. Key Design Decisions

Decision	Rationale
Using Room & LiveData	UI (charts) auto-updates when new data is written or filters change.
Modular Visualization (BaseChartActivity)	Centralizes chart setup logic, reduces code duplication, eases maintenance.
Dynamic Observer Management	In MainGraphActivity, LiveData observers are removed and re-registered on filter change to prevent conflicting updates.
Vector Magnitude Calculation	Sensor intensity is calculated using the Euclidean norm ($\sqrt{X^2 + Y^2 + Z^2}$) instead of simple summation.

3.3. Interfaces / APIs

- Android Notification Manager: Used by SensorEreignis to issue threshold warnings.

3.4. MQTT Broker Setup (Required Before Running App)

- The Android app connects to an MQTT broker running on your PC.
- Follow these steps exactly.

Install Mosquitto Broker (Windows)

- Download from the official website: <https://mosquitto.org/download/>
- Install with default settings

Configure Mosquitto

- Edit (or create):

```
C:\Program Files\mosquitto\mosquitto.conf
```

- Add:

```
listener 1883
allow_anonymous true
```

- This allows your Android phone to connect without username/password.
- Restart Mosquitto after saving the file.

Allow Mosquitto in Windows Firewall

- Open Windows Firewall → “Allow an app” → enable:
 - mosquitto.exe
 - Port 1883/TCP
- If this is blocked, the app will fail with:

```
java.net.SocketTimeoutException: failed to connect
```

Find Your PC’s LAN IP

- Run this in PowerShell or CMD:

```
ipconfig
```

- Look for:

```
IPv4 Address : . . . : 192.168.xxx.xxx
```

- Copy this IP.
- You must put this IP in your Android code:

```
private static final String PHONE_BROKER = "tcp://192.168.xxx.xxx:1883";
```

- This IP must match your PC’s Wi-Fi network.

Phone and PC Must Be on the Same Network

- Your Android phone must be:
 - On Wi-Fi (same router as PC)
 - NOT on mobile data
 - NOT on guest Wi-Fi
 - NOT behind VPN
 - If networks differ → MQTT cannot communicate.

Topic	Meaning	Data Format
Sensor/Bewegung	Acceleration data	x,y,z

Sensor/Gyro	Gyroscope data	x,y,z
Sensor/Zeit	Timestamp string	YYYY-MM-DD...

These are the topics that the application automatically subscribes to after a successful MQTT connection.

3.5. MQTT Connection Workflow

Startup Sequence: 1. Generate a unique MQTT client ID 2. Select the correct broker URL (emulator vs. real device) 3. Instantiate `MqttHandler` 4. Register the message listener callback 5. Connect asynchronously to the broker

On Successful Connection: * Subscribe to all three sensor topics (Sensor/Bewegung, Sensor/Gyro, Sensor/Zeit) * Publish retained initial values for testing * Load historical database entries and log the count

3.6. Processing Incoming Messages

Each topic is processed independently:

- Sensor/Bewegung
 - Payload: x,y,z
 - Parse into float values (X/Y/Z)
 - Update accelerometer TextView
 - Insert values into the database
- Sensor/Gyro
 - Payload: x,y,z
 - Parse into float values (X/Y/Z)
 - Update gyro TextView
 - Insert values into the database
- Sensor/Zeit
 - Payload: x,y,z
 - Parse into float values (X/Y/Z)
 - Update timestamp TextView
 - Insert values into the database

All UI modifications run on the main thread using `runOnUiThread()`.

3.7. Database Integration

Two helper methods handle all Room database operations:

- `storeSensor(ValueSensor sensor)`
 - Runs in a background thread
 - Inserts a new row into the Room DB
 - Logs warnings on failure
- `loadDatabaseValues()`
 - Runs in a background thread
 - Fetches all stored `ValueSensor` rows
 - Logs the total number of entries

3.7.1. **MqttHandler — Responsibilities**

`MqttHandler` is a wrapper for the Eclipse Paho asynchronous MQTT client. It manages:

- Connecting
- Automatic reconnecting
- Subscribing
- Publishing
- Disconnecting
- Forwarding incoming messages

Asynchronous Connect

Features:

- Fully non-blocking
- `automaticReconnect = true`
- `cleanSession = true`

Connection results are delivered via:

- `onConnected()`
- `onFailure(Throwable)`

Subscription

- Uses QoS 1 (“at least once”)
- Logs both success and failure
- Safe to call multiple times

Publishing

Supports:

- Normal transient messages
- Retained messages (retained = true)

Publishing is performed in a background thread to avoid blocking the UI.

Disconnecting

- Gracefully closes the MQTT connection
- Updates internal connected state
- Runs on a background thread
- Prevents crashes when the activity is destroyed

Receiving Messages

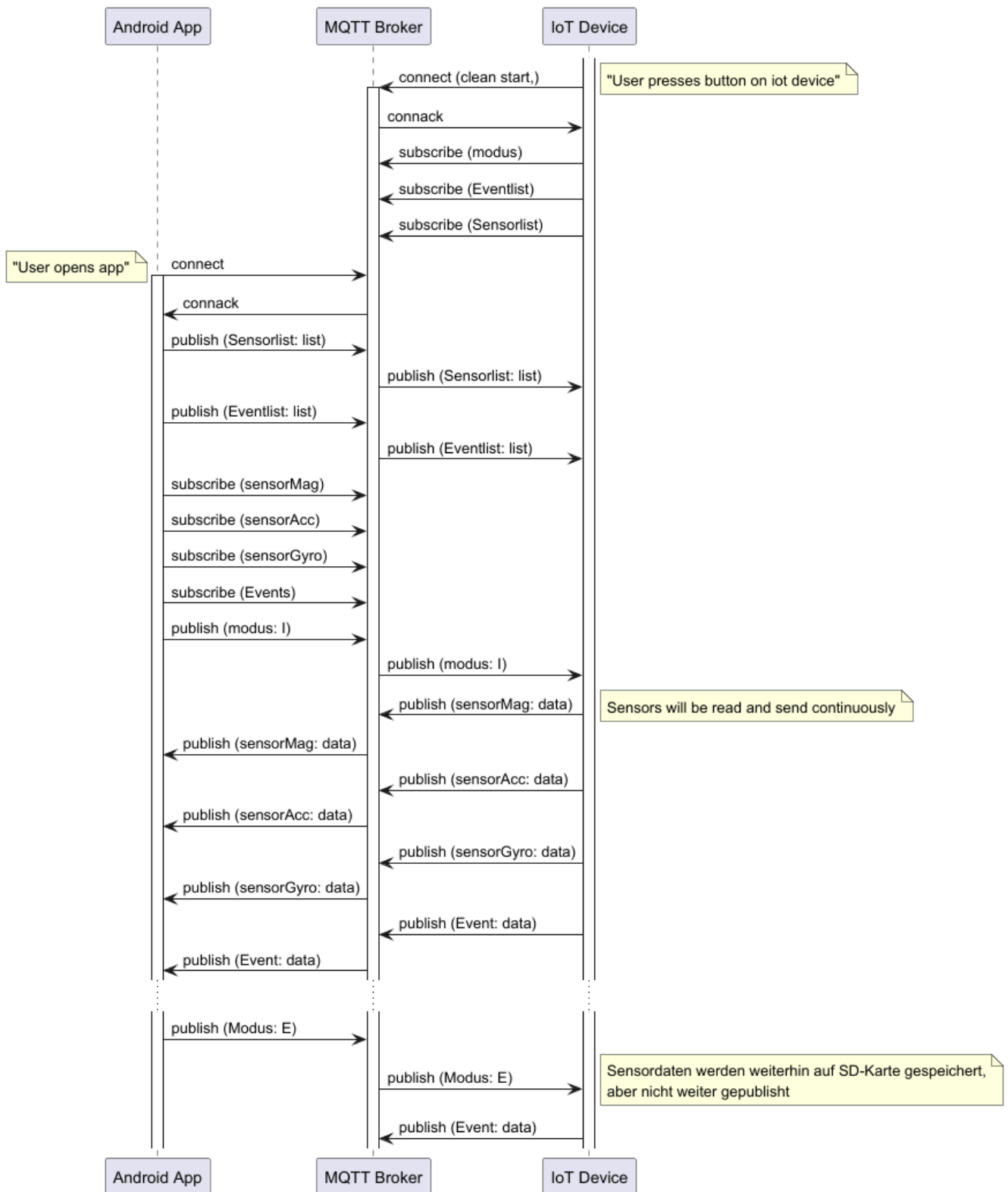
Every MQTT message triggers:

```
onMessageReceived(topic, payload)
```

MainActivity uses this callback to:

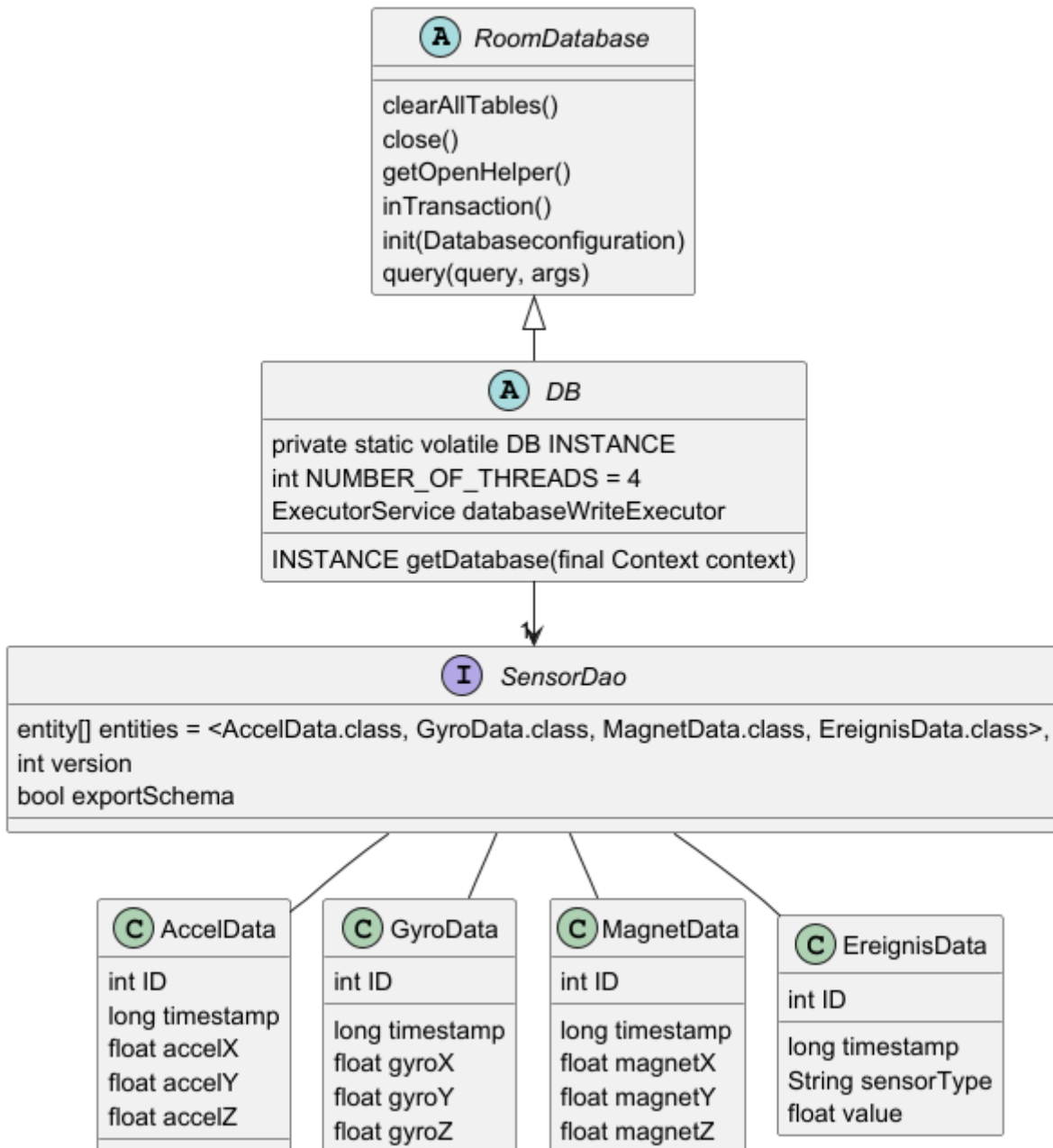
- Update UI fields
- Parse sensor payloads
- Insert data into Room DB

This is the core mechanism for MQTT → Android UI → Database communication.

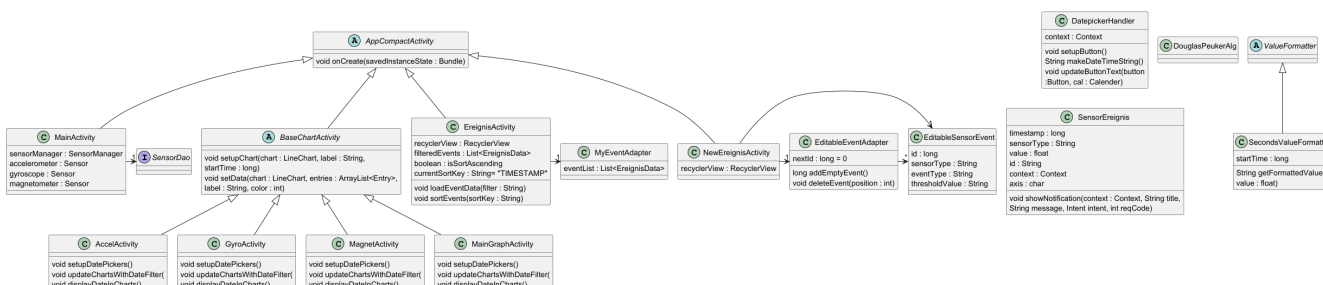


3.8. Events

3.9. DBAccess



3.10. Class Diagram



4. Component and Class Overview

4.1. Main Classes and Responsibilities

- MainActivity: Central controller, UI initialization, MQTT handling, database access
- MqttHandler: MQTT client wrapper, connection management, message receive/send
- BaseChartActivity: Base class for chart views, central chart logic
- AccelActivity, GyroActivity, MagnetActivity: Display and filter respective sensor data
- MainGraphActivity: Overview chart, filtering, LiveData management
- EreignisActivity, NewEreignisActivity: Event management, RecyclerView adapters
- SensorEreignis: Threshold monitoring, notifications
- DatePickerHandler: Date selection for filtering
- DouglasPeukerAlg: Data reduction for charts

4.2. Database Structure

- DB.java: RoomDatabase implementation, singleton, thread pool for writes
- SensorDao, ValueSensorDAO: Access to entities (AccelData, GyroData, MagnetData, EreignisData, ValueSensor)
- Entities: AccelData, GyroData, MagnetData, EreignisData, ValueSensor (each with timestamp and values)

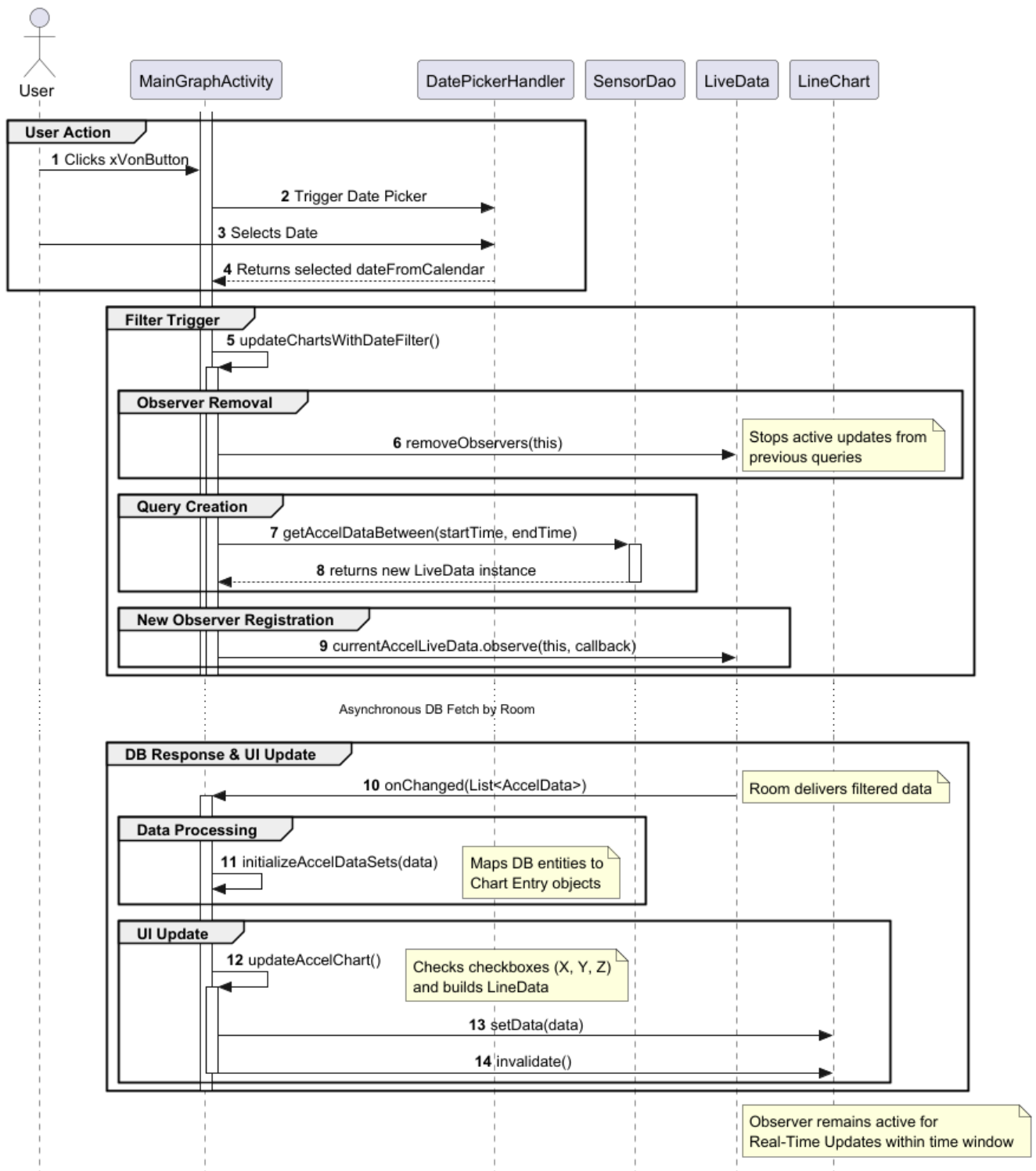
4.3. Simulation and Testing

- SensorDataSimulator: Generates test data for sensors == Runtime & Deployment

5. Runtime & Deployment

- Example flow (one sequence diagram or text description)

Sequence: Manual Chart Filtering in MainGraphActivity



The application is deployed as a standard Android APK and runs locally on the user's mobile device. All data persistence and visualization processing are performed client-side on the device. No external server or cloud is required for core functionality.

5.1. Running the Android App

- The Android app behaves differently depending on where it's running.

5.1.1. Running on an Emulator (Android Studio)

- No network setup required.
- The app automatically uses:

```
tcp://10.0.2.2:1883
```

- Because the emulator cannot access your PC directly — 10.0.2.2 is the gateway.
- MQTT will work immediately if Mosquitto is running on your PC.

5.1.2. Running on a Real Physical Phone

Requirements:

1. Mosquitto must be running on your PC
2. Port 1883 must be open
3. Phone and PC must be on the same Wi-Fi
4. Correct LAN IP set in PHONE_BROKER
5. No VPN, guest network, or firewall blocking

After launching the app: * It will connect to the broker * Subscribe to sensor topics * Publish test messages * Display incoming sensor data * Save it to the local Room database

If connection fails, you will see a Toast:

```
"MQTT error: ..."
```

And the log may show:

```
SocketTimeoutException: failed to connect
```

This always indicates a network/firewall/IP mismatch.

6. Quality & Constraints

Goal	Strategy
Performance (Data Ingestion)	Use DB.databaseWriteExecutor to ensure high-frequency sensor data insertion happens off the main thread, preventing UI freezes.

Performance (Filtering)	Database queries are optimized using WHERE clauses on the indexed timestamp column. Dynamic observer management prevents conflicting asynchronous updates.
Usability (Reactivity)	LiveData ensures charts and data streams update automatically and smoothly, reducing perceived latency for the user.
Usability (Consistency)	BaseChartActivity and DatePickerHandler enforce consistent styling and interaction across the application's filtering interface.

1. Tech Stack Dependence: Strong dependency on Android OS, Java, AndroidX libraries (Room, LiveData), and the external MPAndroidChart library.
2. Hardware Dependency: The core function of the app requires an Android device.

7. How-To (for developers)

7.1. Build & Run Locally

- download app folder from git (https://github.com/FHDW-Hannover-IoT/BioT_Speech_IoT_App.git) (click [main v])
- configure virtual device (we used a Pixel 8a) (Device Manager/ Add a new Device / Create Virtual Device/ Pixel 8a/ Start (Play Button))
- run main class MainActivity (Path: app/kotlin+java/com.fhdw.biot.speech.iot/MainActivity.java, start (PlayButton) first time can take 1 minute or so)
- Android App symbol will be added to the start screen and will open automatically

7.2. Configure (env vars, settings)

Thread Count for Database Writes: The number of threads used for asynchronous database operations is configured in DB.java.

7.3. Testing basics

Class AndererUnitTest

@Before

Creates Context for the notification

- Simulate a Notification Manager to avoid the need for hardware simulation, based on the context of the integrated Notification Service in Android.

testDefineSensorEvent

- Creates a fixed sensor type, a value, an ID, and an axis for the notification. Additionally, use the current time. Finally, create an event with these results. A notification should be triggered immediately after the sensor event is defined, as the class is defined accordingly. Afterwards, the various entered values should be checked for correctness.

testCreateNotification

- Actively creates a notification based on Sensortype, ID, axis and a pre-defined value. The context is simulated. With Verify it is checked if a notification channel has been created. Additionally, the value, ID, axis and importance (Default) will be checked. If these values are correct, then the data for the notification will be too. Afterwards it will be checked if a notification is sent.

DBSchreibenTest: * Creates a new event for the database containing timestamp, sensor type: MAGNET, axis: X, "magEvent_" and context. The databaseWriteExecutor is executed and the event added.

7.4. Testing

This section explains how to verify that your MQTT broker, phone, and Android application all communicate correctly.

7.4.1. Publish From PC (Manual Test)

Use mosquitto_pub from your computer to send test messages directly into the broker.

Example:

```
mosquitto_pub -h 192.168.xxx.xxx -t Sensor/Bewegung -m "1.2,5.6,9.0"
```

Expected behavior:

- The app updates the Bewegung TextView
- A new row is inserted into the Room database
- Logcat shows the incoming message
- No crashes or MQTT disconnects occur

Repeat for the other topics:

```
mosquitto_pub -h 192.168.xxx.xxx -t Sensor/Gyro -m "0.5,1.2,3.4"
mosquitto_pub -h 192.168.xxx.xxx -t Sensor/Zeit -m "2025-01-01T12:00:00Z"
```

7.4.2. Check If the Phone Can Reach the Broker

To verify that your physical Android device can connect to the MQTT broker running on your PC, install any MQTT client app on your phone, such as:

- MQTT Dash
- MQTT Client
- MyMQTT

7.4.3. Test Steps

1. Open the MQTT client app on your phone.
2. Create a new connection using your PC's LAN IP address (the same one used in PHONE_BROKER).
3. Set the port to: 1883
4. Choose protocol: MQTT (TCP)
5. Subscribe to the following topics:
 - Sensor/Bewegung
 - Sensor/Gyro
 - Sensor/Zeit
6. Publish test values from the phone app to confirm two-way communication.

Example test messages:

- Sensor/Bewegung → "1.2,4.5,6.7"
- Sensor/Gyro → "0.1,0.2,0.3"
- Sensor/Zeit → "2025-01-01T12:00:00Z"

7.4.4. Expected Result

If the phone MQTT app can:

- Connect successfully
- Subscribe to topics
- Receive messages
- Publish to the broker

Your network, IP configuration, and Mosquitto setup are correct.

This also confirms:

- If the phone app connects but the Android application does not, the issue is inside the Android app code — not the network or broker.

7.4.5. Extra Debugging Tips

- Make sure your PC firewall allows Inbound TCP 1883
- Ensure your phone and PC are on the same Wi-Fi network

- Test ping:

```
ping 192.168.xxx.xxx
```

- Restart Mosquitto:

```
net stop mosquitto  
net start mosquito
```

- If using a laptop, disable VPNs (they rewrite routes)

8. Open Questions / Risks

- List of things to be clarified or risks to watch

9. Appendix

- References (tickets, ADRs, external docs, etc.)
- [FHDW](#)